Project Phoenix: Comprehensive Engineering, Security, & Observability Standard

By Anoop VK 7 min 27 Add a reaction Karma Page Builder Request approval
Ask to AI ChatGPT Assistant Show less

Version: 5.1

Owner: Cloud Engineering and Security Team

Status: Approved for Implementation

1. Core Principles

1. **Shift Left:** Security is defined in code and checked before commit.
2. **Zero Trust & Zero Access:** No implicit trust between services. No direct access (SSH/RDP) to compute or databases. All interactions occur via code pipelines or observability dashboards.
3. **Serverless First:** Scale-to-Zero capability. Pay only for what is used.
4. **Immutable Artifacts:** Build once, deploy everywhere.
5. **Everything as Code:** Infrastructure, Security Policies, Dashboards, and Alarms are managed via Git.

2. Technology Stack (Atlassian + AWS Native)

We leverage the **Atlassian Open DevOps** toolchain for management and **AWS Native** services for execution.

Domain	Tool/Service	Configuration Notes
Repo & CI/CD	**Bitbucket Cloud & Pipelines**	Single platform for Source Code and CI/CD execution. OIDC for AWS Auth.
Project Mgmt	**Jira Software**	Integrated with Bitbucket. "Deployments" view enabled for tracking.
Incident Mgmt	**Jira Service Mgmt + Opsgenie**	On-call schedules and alert routing.
Documentation	**Confluence**	"Source of Truth" for design docs, runbooks, and API specs.
Catalog	**Atlassian Compass**	Service Registry, Scorecards, and Ownership tracking.
Artifact Repos	**AWS CodeArtifact**	Secure storage for Maven/npm/pip packages (Replacing Nexus).
Container Reg	**Amazon ECR**	Docker image storage with immutable tags.
Compute	**AWS Lambda / ECS Fargate**	ARM64 (Graviton) preferred.
Database	**Aurora Serverless v2 / MongoDB**	PrivateLink connectivity only.
Edge Security	**CloudFront + WAF + Shield**	WAF Mode: Fail Open (initially).
Runtime Security	**AWS GuardDuty**	Intelligent threat detection for Lambda/ECS (Crypto mining, anomalous IP detection).

Compliance Mgmt	**AWS Security Hub**	Centralized dashboard for CSPM (Cloud Security Posture Management) and compliance checks.
GenAI Platform	**AWS Bedrock**	Unified API for all LLM inference (Claude, Titan, etc.) to ensure metering and privacy.
Secret Mgmt	**AWS Secrets Manager**	Rotated automatically. Fetched at runtime.
Observability	**CloudWatch + X-Ray + RUM**	Full stack logging, tracing, and Real User Monitoring.

3. API Performance Standards (NFRs)

To ensure a world-class user experience, all APIs must meet the following Non-Functional Requirements (NFRs).

3.1 Latency Targets (Server-Side)

- **P95 Response Time:** < **200ms** for standard read/write APIs.
- **P99 Response Time:** < **500ms** (No "long tail" outliers).
- **Cold Start:** < **2 seconds** (Mandatory use of AWS SnapStart for Java).

3.2 Global Edge Performance

- **Content Delivery:** Static assets (JS/CSS/Images) must be served from **CloudFront Edge locations** with a cache hit ratio target of **> 90%**.
- **TTFB (Time to First Byte):** < **50ms** for cached content.

3.3 Throughput & Scalability

- **Concurrency:** Services must scale horizontally to handle **10x** standard load.
- **Rate Limiting:** All Public APIs must have throttling enabled via API Gateway.

3.4 Service Health Endpoint Standard (Mandatory)

To support canary rollouts and rapid incident triage, every service must implement a standard health endpoint.

3.4.1 Endpoint Requirements

- **Path:** `GET /<applicationPath>/health`
  Eg: /workflow-lite/health, /data/health, /drive/health
- **Response Format:**
  - All services must return **HTTP 200** when healthy and respond with the following JSON payload:

```
{
  "message": "app is healthy",
  "service": "<servicename>",
  "status": "healthy",
  "version": "<version from version.txt>",
  "additional": {}
}
```

Required fields

- `message` (string): must be something like `"app is healthy"`
- `service` (string): service name (Should be same as Bitbucket repo name)
- `status` (string): must be `"healthy"` for healthy state
- `version` (string): must be read from `version.txt` packaged with the artifact
- `additional information:` Teams may include extra keys for troubleshooting, but they must be:
  - non-sensitive
  - small (avoid large payloads)

- stable across releases

4. API Governance & Design Standards

To prevent "API Sprawl" and ensure consistency across microservices.

- **Versioning:** URI-based versioning is mandatory (e.g., `/v1/users`). Breaking changes require a new version (e.g., `/v2/users`).
- **Documentation:** All services must maintain an **OpenAPI 3.0 (Swagger)** specification in the repository.
  - *Governance:* **Atlassian Compass** will automatically render these specs from the repo.
- **Standard Error Envelope:** All APIs must return errors in a strict JSON format:

```JSON
{
  "error_code": "PHX-ERR-001",
  "message": "Resource not found",
  "trace_id": "1-5f..."
}
```

5. Cloud Architecture & Cost Optimization

5.1 Compute & Networking

- **Container Constraints:**
  - **Size:** Limit of **100MB - 150MB** (Distroless or Alpine base).
  - **Boot Time:** Optimized for scale-to-zero efficiency.
- **API Segregation:**
  - **Public:** CloudFront > WAF > API Gateway (Public).
  - **Internal:** Private API Gateway (VPC Endpoint) for Service-to-Service.
- **Scale-to-Zero:** Non-prod environments must scale down to 0 instances when idle.

5.2 Storage & Data

- **Customer Data:** S3 with Lifecycle Policies (Transition to Glacier/Deep Archive).
- **Database:** Aurora Serverless v2 scaling ACUs dynamically.

6. The DevSecOps Pipeline (Bitbucket Pipelines)

Step 1: Local Development

- **Requirement:** Developers use **SonarLint** and **Trivy Plugin** in IDE. No commits allowed with critical linting errors.

Step 2: Continuous Integration (On PR)

1. **Build & Test:** Run Unit Tests (Target: >80% coverage).
2. **SAST: SonarCloud** scan (Quality Gate: Strict).
3. **SCA (Supply Chain):**
   - Generate SBOM (CycloneDX) & Upload to **Dependency Track**.
   - Check for vulnerabilities in **AWS CodeArtifact**.
4. **Artifact Build:** Create Docker Image.
5. **Container Scan: Trivy** scan (Blocker: Critical OS vulnerabilities).
6. **Publish:** Push to **Amazon ECR** (Immutable Tags).

Step 3: Deployment (GitOps)

- **Tool: Terraform** (executed via Bitbucket Pipelines).
- **Process:** Pipeline updates the infrastructure state. No manual console changes.
- **Catalog Update:** Webhook updates **Atlassian Compass** deployment history.

Step 4: Security Verification (DAST)

- **Tool: Beagle Security**.
- **Action:** Automated penetration test against Staging API.
- **Gate:** Block Production promotion if High/Critical issues found.

7. Deployment Strategy: Canary & Rollbacks

We use **AWS CodeDeploy** for zero-downtime releases.

7.1 The Canary Process

1. **Shift:** Route **10%** traffic to the new version.
2. **Wait:** Monitor health for **X minutes**.
3. **Promote:** If healthy, route **100%**.

7.2 Automated Rollback (Kill Switch)

Rollback is fully automated based on CloudWatch Alarms.

Metric	Threshold	Response Time
Error Rate (5xx)	> 1% of requests	< 1 Minute
Latency (P95)	> 500ms (or SLA)	< 2 Minutes
App Crash	Log keyword "Panic"	Instant

8. Incident Management (Opsgenie)

Alerts must be actionable and routed to the correct owners immediately.

- **Tooling: Atlassian Opsgenie** (integrated with Jira Service Management).
- **Workflow:**
  a. **Detection:** CloudWatch Alarm triggers (e.g., "Latency High").
  b. **Routing:** Opsgenie routes the alert to the **On-Call Engineer** defined in the Compass Team roster.
  c. **Ticket:** A Jira Incident ticket is created automatically.
  d. **Resolution:** Post-mortem report created in Confluence linked to the Jira ticket.

9. Disaster Recovery & Business Continuity

- **RPO (Data Loss Tolerance):** < 15 Minutes.
- **RTO (Downtime Tolerance):** < 4 Hours.
- **Database Strategy:**
  - Automated daily snapshots (Retention: 30 days).
  - Cross-Region Replication enabled for critical datasets.
- **Drill Frequency:** DR "Game Days" conducted bi-annually to test failover scripts.

10. Data Residency & Compliance

- **Data Residency:** Customer data is pinned to the contractual AWS Region. No unauthorized cross-border replication.
- **Encryption:**
  - **At Rest:** AES-256 via AWS KMS (CMK).
  - **In Transit:** TLS 1.3 mandatory for all internal and external flows.
- **Audit Trails: AWS CloudTrail** enabled for all regions. Logs stored in a WORM (Write Once Read Many) S3 bucket for legal hold.

11. Security & Operations (Zero Touch)

11.1 Authentication & Secrets

- **Pipeline Auth:** Bitbucket Pipelines connects to AWS via **OIDC** (No long-lived IAM keys).
- **Runtime Secrets:** Fetched from **AWS Secrets Manager** at boot.

11.2 Vulnerability Management

- **Weekly API Pentesting:** Automated API Penetration Testing (via Beagle Security) must run weekly on Production.

11.3 Observability Stack

- **Logs: Amazon CloudWatch Logs** (Structured JSON).
- **Tracing: AWS X-Ray** for distributed tracing.
- **RUM: AWS CloudWatch RUM** for client-side performance monitoring.
- **Access Control:** No SSH/RDP. All debugging is done via Dashboards.

11.4 Structured Logging Standard (Mandatory)

Goal: Every production issue must be traceable from the origin request to all downstream sub-requests (HTTP, DB, queues). This requires JSON structured logs with consistent identifiers across services.

Rule 1 — JSON-only logs:
All application logs must be emitted as one JSON object per log line (no multiline logs).

Rule 2 — Correlation + Tracing required:
Every log entry must include:

- `requestId` (unique per service/request instance, e.g., API GW / Lambda request id or generated)
- `trace.traceId`, `trace.spanId`, `trace.parentSpanId` (for call-tree reconstruction)

Rule 3 — HTTP + duration standardization:
For request/response logs, always include:

- `method`, `path`, `status`, `durationMs`

Rule 4 — Tenant context:
All logs must carry tenant and application context:

- x-jiffy-tenant-name
- x-jiffy-app-name

Application Log Format (v1)

```
{
  "ts": "2025-12-23T09:10:11.123Z",
  "level": "INFO",
  "serviceName": "app-manager",
  "serviceVersion": "1.1.7",
  "log": {
    "message": "Human readable message",
    "event": "business|security|dependency|audit|system",
    "status": "success|failure|unknown"
  },
  "requestId": "xxxx-xxxx-xxxx-xxxxxx",
  "method": "GET",
  "path": "/health",
  "status": 200,
  "durationMs": 23,
  "trace": {
    "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
    "spanId": "00f067aa0ba902b7",
    "parentSpanId": "def456"
  },
  "aws": {
    "cfId": "x-amz-cf-id",
    "apiGwRequestId": "context.requestId",
    "apiGwExtendedRequestId": "context.extendedRequestId",
    "xrayTraceId": "Root=1-..."
  },
```

```
  "jiffy": {
    "headers": {
      "x-jiffy-tenant-name": "t-123",
      "x-jiffy-app-name": "56778",
    }
  }
}
```

Trace ID should be the same across a request and its sub calls.

Required Fields

Always required (all log lines):

- `ts`, `level`, `name`, `version`, `log`
- `requestId`
- `trace.traceId`, `trace.spanId`
- `x-jiffy-tenantid` and `x-jiffy-apid`

Required for HTTP request lifecycle logs:

- `method`, `path`, `status`, `durationMs`

Required for errors (in addition to above):
Add an `error` object and set `level=ERROR`:

12. Generative AI & LLM Governance

To mitigate data exfiltration risks and ensure centralized cost tracking ("Shadow AI"), all Generative AI workloads must adhere to this standard.

- **Unified Platform: AWS Bedrock** is the **sole authorized provider** for LLM inference (e.g., Claude, Titan, Llama). Direct use of external API keys (OpenAI, Anthropic, etc.) in code is **strictly prohibited**.
- **Privacy & Connectivity:**
  - **PrivateLink:** All Bedrock calls must be routed via VPC Endpoints. Traffic must never traverse the public internet.
  - **Data Usage:** Training on customer data is explicitly disabled at the Organization level.
- **Cost Attribution:**
  - All AI model invocation must be tagged with `CostCenter` and `ProjectID`.
  - **CloudWatch** alarms must be configured to trigger at 80% of the daily spend quota for inference.

13. Cost Management

- **Tagging:** Mandatory `CostCenter (Department/Customer)`, `ServiceOwner`, `Environment` tags.
- **Budgets:** AWS Budgets configured to alert if forecasted spend > 80%.
- **Spot Strategy:** CI/CD runners and non-production workloads must use Spot Instances.

14. Infrastructure Provisioning

1. **Infrastructure ID (Infraid) Convention**
   a. Format: PPX0YY

Position	Meaning	Values
PP	Product Code	2-letter alphabetical code
X	Environment Type	1 = Non-prod, 2 = Prod, 3 = DR/Backup
YY	Sequence	01-99

Sample: `PP1001`

2. **Tagging**

```
Product      = "phoenix"
Environment  = "dev"
Contact      = "devops@jiffy.ai"
Department   = "phoenix-engg"
ServiceOwner = "engineering"
Customer     = "internal"
Infraid      = "PP1001"
Project      = "jiffy-phoenix"
ManagedBy    = "terraform"
```

   a. Customer Tag values:
      i. `shared` : Infrastructure shared by all customers (VPC, NAT Gateway, Transit Gateway)
      ii. `platform` : Platform/internal services (CI/CD, Monitoring, Logging)
      iii. `internal` : Internal company use only (Dev environments, Sandbox)
      iv. `{customer-name}` : Customer dedicated resources
   b. ServiceOwner: ?
      Eg: Engineering
   c. Department: ?
      Eg: phoenix-dev